Programming project 5: Operator Overload

Start Assignment

- Due Apr 9 by 11:59pm
- Points 100
- Submitting a file upload
- File Types cc and h
- Available until Apr 12 at 11:59pm

`HugeNumber` Class

Write a C++ class named `HugeNumber` that represents an integer of **arbitrary length** without using built-in large integer libraries.

Your class must internally store the number as a **sequence of decimal digits**, using a `vector`. The class must work for values much larger than `long long`.

Internal representation

You may choose one of the following designs:

- store digits in a `vector<int>`
- store digits in a `vector<char>`

But in all cases:

- the sign must be stored **separately** as a boolean or equivalent field
- you may **not** encode the sign as a negative digit inside the digit container

This is an intentional change from prior versions of similar assignments.

Required constructors

Your class must support the following constructors:

1. **Default constructor**
   - creates the value `0`
2. **Constructor from** `std::string`
   - examples:
     - `"12345678901234567890"`
     - `"-45009"`
3. **Constructor from a character array and a size**

- example:
  - `HugeNumber a(arr, n);`

4. **Constructor from a vector of digits**
   - example:
     - `vector<int> d = {1,2,3,4,5};`
     - creates `12345`

For the vector constructor:

- the digits vector contains **only digits**
- sign is passed as a **second argument**
- example:
  - `HugeNumber({1,2,3,4}, true)` means negative
  - `HugeNumber({1,2,3,4}, false)` means positive

Required operator overloads

Overload the following operators for `HugeNumber`:

- `+`
- `-`
- `*`
- `==`
- `!=`
- `<`
- `<=`
- `>`
- `>=`
- prefix `++`
- postfix `++`
- prefix `--`
- postfix `--`
- `<<`
- `>>`

Important redesign note

Instead of overloading `!` as in older versions, this assignment requires:

New unary operator

- overload unary `~`

- `~x` must return a new `HugeNumber` with its digits reversed

Examples:

- `~12345` becomes `54321`
- `~-9080` becomes `-809`

This gives you another operator-overloading target while making old solutions less reusable.

Additional required member functions

Implement these methods:

1. `bool isZero() const`
2. `int numDigits() const`
3. `HugeNumber abs() const`
4. `void normalize()`
   - removes leading zeros
   - ensures zero is never stored as negative

Input validation rules

Your constructors must reject invalid input such as:

- empty input
- non-digit characters other than a leading `-`
- malformed arrays

You may:

- throw an exception, or
- print an error and terminate

State your choice in comments.

Behavioral requirements

1. Arithmetic must work correctly for:
   - positive numbers
   - negative numbers
   - zero
   - values of different lengths

2. Comparison operators must work correctly for all sign combinations.
3. Prefix/postfix increment and decrement must behave exactly like built-in integers.
4. `<<` must print the number in normal readable form.
5. `>>` must read a number from input and build a valid `HugeNumber`.

Prohibited shortcuts

You may **not** use:

- `boost::multiprecision`
- conversion of the whole value into `long long`, `int128`, or similar built-in large numeric types
- string-to-integer conversion for the full arithmetic logic

You must operate on the stored digits directly.

Testing requirements

In `main()`, demonstrate all of the following:

1. construction from:
   - string
   - char array
   - vector of digits
2. arithmetic with:
   - two positive numbers
   - two negative numbers
   - one positive and one negative
3. comparisons between unequal and equal values
4. prefix and postfix `++` / `--`
5. digit-reversal operator `~`
6. stream insertion and extraction using `<<` and `>>`

Example test cases

Your program output should clearly show tests similar to these:

- `999999999999 + 1`
- `1000000000000 - 1`
- `-2500 + 300`

- `-25 * -4`
- `-25 * 4`
- `~123400`
- `++x`, `x++`, `--x`, `x--`
- comparisons such as:
  - `12345 > 12344`
  - `-999 < 5`
  - `-100 == -100`

Submission requirements

Submit:

1. `HugeNumber.h`
2. `HugeNumber.cpp`
3. `main.cpp`
4. a short README explaining:
   - your internal representation
   - how sign is stored
   - how addition/subtraction/multiplication are implemented

Grading rubric

- Constructors and normalization: **15 pts**
- Comparison operators: **15 pts**
- Addition and subtraction: **20 pts**
- Multiplication: **20 pts**
- Increment/decrement: **10 pts**
- Stream operators: **10 pts**
- Unary `~` operator: **5 pts**
- Code quality / style / decomposition: **5 pts**